

5–7 LPA Placement Preparation

Questions 1–15 — Brute Force + Optimized C++

Built-in reverse/sort methods use nahi kiye gaye hain. Har question me approach, C++ code, logic/dry run aur time/space complexity di gayi hai.

1. Reverse a String

Example: hello → olleh

Brute Force – Extra String:

```
string s = "hello";
string rev = "";
for(int i = s.length() - 1; i >= 0; i--) {
    rev = rev + s[i];
}
cout << rev;
```

Logic: Last index se first index tak characters ko naye string me add karte hain.

Complexity: Time $O(n)$, Space $O(n)$.

Optimized – Two Pointer:

```
string s = "hello";
int left = 0, right = s.length() - 1;
while(left < right) {
    char temp = s[left];
    s[left] = s[right];
    s[right] = temp;
    left++;
    right--;
}
cout << s;
```

Logic: First aur last character swap karte hain, phir pointers andar move hote hain.

Complexity: Time $O(n)$, Space $O(1)$.

2. Check if a String is a Palindrome

Brute Force:

```
string s = "madam";
string rev = "";
for(int i = s.length() - 1; i >= 0; i--)
    rev = rev + s[i];
```

```
if(s == rev) cout << "Palindrome";
else cout << "Not Palindrome";
```

Optimized – Two Pointer:

```
string s = "madam";
int left = 0, right = s.length() - 1;
bool palindrome = true;
```

```
while(left < right) {
    if(s[left] != s[right]) {
        palindrome = false;
        break;
    }
    left++;
    right--;
}
```

```
if(palindrome) cout << "Palindrome";
else cout << "Not Palindrome";
```

Logic: First-last, second-second-last characters compare karte hain.

Complexity: Brute $O(n)$ time/ $O(n)$ space; Optimized $O(n)$ time/ $O(1)$ space.

3. Check if a Number is a Palindrome

Example: 121 → Palindrome

```

Approach – Reverse Number:
int n = 121;
int original = n, rev = 0;

while(n > 0) {
    int digit = n % 10;
    rev = rev * 10 + digit;
    n = n / 10;
}

if(original == rev) cout << "Palindrome";
else cout << "Not Palindrome";

```

Dry run: 121 → digit 1, rev 1 → digit 2, rev 12 → digit 1, rev 121.

Complexity: Time $O(\log n)$, Space $O(1)$.

Advanced optimization: Sirf half number reverse karke compare kiya ja sakta hai, lekin normal reverse method interview me explain karna easiest hai.

4. Reverse an Integer

Example: 12345 → 54321

```

int n = 12345;
int rev = 0;

while(n > 0) {
    int digit = n % 10;
    rev = rev * 10 + digit;
    n = n / 10;
}
cout << rev;

Important formula:
digit = n % 10;
rev = rev * 10 + digit;
n = n / 10;

```

Complexity: Time $O(\log n)$, Space $O(1)$.

Overflow-safe interview version:

```

if(rev > INT_MAX / 10 || rev < INT_MIN / 10)
    // overflow

```

5. Check if a Number is Prime

```

Brute Force:
int n = 29;
bool prime = true;

if(n <= 1) prime = false;

for(int i = 2; i < n; i++) {
    if(n % i == 0) {
        prime = false;
        break;
    }
}

if(prime) cout << "Prime";
else cout << "Not Prime";

```

Complexity: $O(n)$.

```

Optimized –  $\sqrt{n}$ :
for(int i = 2; i * i <= n; i++) {
    if(n % i == 0) {
        prime = false;
        break;
    }
}

```

Why $\sqrt{n}$? Agar n composite hai, to uska ek factor $\sqrt{n}$ se chhota/equal zaroor hota hai.

Complexity: $O(\sqrt{n})$, Space $O(1)$.

6. Print Prime Numbers in a Range

Brute Force:

```
int start = 1, end = 20;
for(int n = start; n <= end; n++) {
    if(n < 2) continue;
    bool prime = true;
    for(int i = 2; i < n; i++) {
        if(n % i == 0) {
            prime = false;
            break;
        }
    }
    if(prime) cout << n << " ";
}
```

Optimized – $\sqrt{n}$:

```
for(int n = start; n <= end; n++) {
    if(n < 2) continue;
    bool prime = true;
    for(int i = 2; i * i <= n; i++) {
        if(n % i == 0) {
            prime = false;
            break;
        }
    }
    if(prime) cout << n << " ";
}
```

Large range ke liye Sieve of Eratosthenes:

```
bool isPrime[21];
for(int i = 0; i <= 20; i++) isPrime[i] = true;
isPrime[0] = isPrime[1] = false;

for(int i = 2; i * i <= 20; i++) {
    if(isPrime[i]) {
        for(int j = i * i; j <= 20; j += i)
            isPrime[j] = false;
    }
}

for(int i = 2; i <= 20; i++)
    if(isPrime[i]) cout << i << " ";
```

Complexity: Individual $\sqrt{n}$ checking $\approx O(n\sqrt{n})$; Sieve $O(n \log \log n)$ time, $O(n)$ space.

7. Find Factorial of a Number

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Iterative:

```
int n = 5;
long long fact = 1;

for(int i = 1; i <= n; i++)
    fact = fact * i;

cout << fact;
```

Complexity: $O(n)$ time, $O(1)$ space.

Recursive:

```
long long factorial(int n) {
    if(n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}
```

Recursive approach $O(n)$ time and $O(n)$ stack space. Exact factorial calculate karne ke liye asymptotically better simple multiplication approach nahi hai.

8. Generate Fibonacci Series

Series: 0 1 1 2 3 5 8 13...

Brute Force – Recursion:

```
int fib(int n) {
```

```

if(n == 0) return 0;
if(n == 1) return 1;
return fib(n - 1) + fib(n - 2);
}

for(int i = 0; i < n; i++)
cout << fib(i) << " ";

```

Problem: Same values baar-baar calculate hoti hain. Time $\approx O(2^n)$.

```

Optimized – Iterative:
int a = 0, b = 1;
for(int i = 0; i < n; i++) {
cout << a << " ";
int next = a + b;
a = b;
b = next;
}

```

Complexity: $O(n)$ time, $O(1)$ space.

9. Find the Nth Fibonacci Number

Assume $F(0)=0$ and $F(1)=1$. Example $F(6)=8$.

```

Brute Force:
int fib(int n) {
if(n == 0) return 0;
if(n == 1) return 1;
return fib(n - 1) + fib(n - 2);
}
cout << fib(6);

```

Time $\approx O(2^n)$.

```

Optimized:
int n = 6;
if(n == 0) cout << 0;
else {
int a = 0, b = 1;
for(int i = 2; i <= n; i++) {
int next = a + b;
a = b;
b = next;
}
cout << b;
}

```

Complexity: $O(n)$ time, $O(1)$ space.

10. Check if a Number is Armstrong

Example: $153 = 1^3 + 5^3 + 3^3 = 153$.

```

General brute-force method (pow built-in nahi):
int n = 153;
int original = n, temp = n;
int digits = 0;

while(temp > 0) {
digits++;
temp /= 10;
}

int sum = 0;
temp = n;
while(temp > 0) {
int digit = temp % 10;
int power = 1;
for(int i = 0; i < digits; i++)
power *= digit;
sum += power;
temp /= 10;
}

```

```

if(sum == original) cout << "Armstrong";
else cout << "Not Armstrong";

```

Complexity for d digits: $O(d^2)$ time.

```

For specifically 3-digit Armstrong numbers:
while(n > 0) {
int digit = n % 10;
sum += digit * digit * digit;
n /= 10;
}
Complexity  $O(d)$  for this fixed-power case.

```

11. Check if a Number is Perfect

Perfect number: Proper divisors ka sum = number.

Example: $6 \rightarrow 1+2+3=6$.

```

Brute Force:
int n = 28, sum = 0;
for(int i = 1; i < n; i++)
if(n % i == 0) sum += i;

if(sum == n) cout << "Perfect Number";
else cout << "Not Perfect";

```

Complexity: $O(n)$.

```

Optimized – Divisor Pairs:
int n = 28;
if(n <= 1) cout << "Not Perfect";
else {
int sum = 1;
for(int i = 2; i * i <= n; i++) {
if(n % i == 0) {
sum += i;
if(i != n / i) sum += n / i;
}
}
if(sum == n) cout << "Perfect Number";
else cout << "Not Perfect";
}

```

Why? Divisors pairs me aate hain: $28 \rightarrow (1,28), (2,14), (4,7)$.

Complexity: $O(\sqrt{n})$, $O(1)$ space.

12. Find GCD of Two Numbers

```

Brute Force:
int a = 12, b = 18;
int gcd = 1;
int limit = (a < b) ? a : b;

for(int i = 1; i <= limit; i++)
if(a % i == 0 && b % i == 0)
gcd = i;

cout << gcd;

```

Complexity: $O(\min(a,b))$.

```

Optimized – Euclidean Algorithm:
int a = 12, b = 18;

while(b != 0) {
int remainder = a % b;
a = b;
b = remainder;
}

cout << a;

```

Dry run:

$\text{gcd}(18,12) \rightarrow \text{gcd}(12,6) \rightarrow \text{gcd}(6,0) \rightarrow 6$.

Complexity: $O(\log(\min(a,b)))$.

13. Find LCM of Two Numbers

Brute Force:

```
int a = 12, b = 18;
int lcm = (a > b) ? a : b;

while(true) {
    if(lcm % a == 0 && lcm % b == 0)
        break;
    lcm++;
}
cout << lcm;
```

Optimized — GCD Formula:

$LCM(a,b) = (a / GCD(a,b)) * b$

```
int gcd(int a, int b) {
    while(b != 0) {
        int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

int a = 12, b = 18;
int g = gcd(a, b);
int lcm = (a / g) * b;
cout << lcm;
```

$(a/g)*b$ is used to reduce overflow risk compared with $(a*b)/g$.

Complexity: $O(\log(\min(a,b)))$.

14. Count Digits in a Number

Example: 12345 $\rightarrow$ 5 digits.

Approach:

```
int n = 12345;
int count = 0;

while(n > 0) {
    count++;
    n = n / 10;
}

cout << count;
```

Dry run:

12345 $\rightarrow$ 1234 $\rightarrow$ 123 $\rightarrow$ 12 $\rightarrow$ 1 $\rightarrow$ 0

Count = 5.

Special case: $n=0$ means 1 digit.

Mathematical alternative:

```
int count = floor(log10(n)) + 1;
```

Lekin interview me repeated /10 method simple aur safe hai.

Complexity: $O(\log_{10} n)$ time, $O(1)$ space.

15. Sum of Digits of a Number

Example: 12345 $\rightarrow$ 1+2+3+4+5 = 15.

Iterative:

```
int n = 12345;
int sum = 0;

while(n > 0) {
    int digit = n % 10;
    sum = sum + digit;
    n = n / 10;
}

cout << sum;
```

Dry run:

12345 → digit 5, sum 5

1234 → digit 4, sum 9

123 → digit 3, sum 12

12 → digit 2, sum 14

1 → digit 1, sum 15

Complexity: $O(\log n)$ time, $O(1)$ space.

Recursive:

```
int sumDigits(int n) {  
    if(n == 0) return 0;  
    return (n % 10) + sumDigits(n / 10);  
}
```

Recursive approach uses $O(\log n)$ stack space.

■ Interview Revision

Two Pointer → Q1, Q2

$\sqrt{n}$ Optimization → Q5, Q6, Q11

Euclidean Algorithm → Q12, Q13

Digit Extraction: $\text{digit} = n \% 10$; $n = n / 10$ → Q3, Q4, Q10, Q14, Q15

Recursion vs Iteration → Q7, Q8, Q9, Q15

Sieve of Eratosthenes → Q6 for a large range